密证 CredProof

AI 生成安全补丁的可信验收与人机协同修复平台

评委操作手册

参赛队伍：DUIDUI 队

作品类别：第五届中国研究生网络安全创新大赛 · 创意作品赛

文档版本：V1.0

发布日期：2026 年 10 月

目　录

1 概述.....3
1.1 文档用途.....3
1.2 交付内容.....3
1.3 运行模式.....3
1.4 评审操作流程.....4
2 启动与停止.....4
2.1 Windows 查看模式.....4
2.2 停止程序.....5
2.3 Windows 隔离验收模式.....5
2.4 Linux 运行.....6
2.5 确认运行环境.....6
3 界面说明.....7
3.1 主界面布局.....7
3.2 功能页面一览.....7
4 首页案例.....8
5 新建检查.....9
5.1 选择项目.....9
5.2 填写检查配置.....10
5.3 冻结配置并开始.....12
6 项目页操作.....12
6.1 打开项目.....12
6.2 运行检查与阅读结果.....13
6.3 修订代码.....14
6.4 人工决定.....15
6.5 导出.....16
7 复检导出包.....16
8 参考材料页面.....18
9 命令行操作.....18
10 常见问题.....19
附录 A 14 项必要条件.....19
附录 B 结论与退出码.....20

1 概述

1.1 文档用途

本手册说明密证 CredProof 作品交付包的启动方法和各功能页面的操作步骤，供评审专家查看、操作和验证作品时使用。手册中的界面截图和示例数据均来自交付包在隔离验收模式下的实际运行。

密证 CredProof 是面向 Python 工具安全修改的验收平台：它把修改后的代码放进一次性隔离副本中真实运行，检查凭据是否泄露、文件是否越界读取、请求是否发往未授权服务，同时确认必要的业务行为仍然成立，给出“通过 / 不通过 / 证据不足”三值结论，再由负责人决定是否采纳。

1.2 交付内容

表 1　交付包主要文件

文件或目录	说明
密证 CredProof.exe	Windows 启动程序，与 start-credproof.cmd 等价，以查看模式启动
start-credproof.cmd	Windows 查看模式启动脚本，使用随包 Python
start-recheck.cmd	Windows 隔离验收模式启动脚本，需要已准备的 WSL2 隔离环境
start-live.cmd	Windows 现场模式启动脚本，需要隔离环境和本地模型
start-credproof.sh	Linux / macOS 查看模式启动脚本
runtime/python/	随包的官方 Windows 嵌入式 Python 3.12.10
README.md	源码说明、各系统环境依赖、模型下载与安装方法
项目简介.pdf、评委操作手册.pdf	作品简介与本手册
examples/	示例项目：资料助手（问题版本与修复版本）、8 个对照评测目标、python-dotenv 接入示例
credproof_safety/、credproof_access/、agent_pilot/	判定核心、受控访问组件、工作台服务与前端源码

1.3 运行模式

同一套界面有三种运行模式，功能权限由后端按模式控制。所有模式都只监听本机地址 127.0.0.1，不上传代码，不调用付费接口。

表 2　三种运行模式

模式	启动方式	运行条件	可用功能
查看	密证 CredProof.exe 或 start-credproof.cmd；Linux / macOS 为 sh start-credproof.sh	Windows 无需安装任何软件；Linux / macOS 需 Python 3.11 及以上	阅读全部已保存记录；接入项目、填写配置、修订代码、记录人工决定。运行检查和复检不可用
隔离验收	start-recheck.cmd，或命令行 --mode recheck	Linux 隔离运行时（Windows 上为 WSL2 中的 Ubuntu 24.04）	以上全部，加上隔离检查、修订后重新检查、复检导出包；不调用模型
现场	start-live.cmd，或命令行 --mode live	隔离运行时，以及本地 Ollama 和 Qwen 模型	以上全部，加上“请本地模型提出候选”

评审时一般先用查看模式浏览全部页面；需要亲自运行检查时，按第 2.3 节或第 2.4 节准备隔离环境后使用隔离验收模式。第 5 章至第 7 章的检查、复检操作需要隔离验收模式。

1.4 评审操作流程

建议按以下顺序评审，各步骤对应的章节见括号：

1. 启动程序，确认页面右上角的运行模式（第 2 章）。
2. 阅读首页案例的六个步骤，了解产品要解决的问题（第 4 章）。
3. 在“新建检查”中选择示例项目“资料助手（问题版本）”，查看配置向导并冻结配置（第 5 章）。
4. 在项目页运行检查、阅读结果、修订代码、重新检查、做出人工决定并导出（第 6 章，需隔离验收模式）。
5. 把导出包上传到“复检导出包”，在新目录重新检查（第 7 章，需隔离验收模式）。
6. 查阅参考材料页面中的对照评测、补丁记录和流水线记录（第 8 章）。

2 启动与停止

2.1 Windows 查看模式

适用于 Windows 10 / 11（64 位），无需安装 Python 或其他软件。

1. 将交付压缩包**完整解压**到本地目录。解压路径可以包含中文或空格，但不要只拷贝启动文件。

2. 双击解压目录中的 **密证 CredProof.exe**（或 start-credproof.cmd）。
3. 若系统弹出“Windows 已保护你的电脑”，点击“更多信息”，再点击“仍要运行”。
4. 程序打开一个命令行窗口，显示本机访问地址（例如 http://127.0.0.1:8765），并自动用默认浏览器打开该地址。
5. 若浏览器未自动打开，将窗口中显示的地址复制到浏览器地址栏。默认端口 8765 被占用时，程序会自动改用空闲端口，以窗口中显示的地址为准。

注：推荐使用 Chrome 或 Edge 浏览器。查看模式启动后，页面右上角显示“只读模式：不执行检查”。

2.2 停止程序

在命令行窗口中按 Ctrl+C，或直接关闭该窗口。程序产生的数据保存在解压目录下的 runs 目录中；删除该目录即可清除全部本机记录。查看模式下打开的演示审阅记录是临时副本，程序退出时自动关闭。

2.3 Windows 隔离验收模式

隔离检查在 WSL2 中的 Ubuntu 24.04 内执行，Windows 只运行工作台。准备步骤如下：

1. 以管理员身份打开 PowerShell，安装 WSL2 与 Ubuntu 24.04：

```
wsl --install -d Ubuntu-24.04
```

2. 在 Ubuntu 中安装系统组件并准备隔离运行时（系统 python3 须为 Ubuntu 24.04 自带的 3.12）。其中 <解压目录> 替换为交付包在 WSL 中的路径，例如 /mnt/e/密证 CredProof：

```
sudo apt-get install -y python3 python3-venv bubblewrap ca-certificates libseccomp2
cd "<解压目录>"
python3 scripts/runtime/setup-linux.py --runtime-root ~/credproof-agent-runtime
CREDPROOF_RUNTIME_ROOT=~/credproof-agent-runtime python3 -m credproof_safety doctor
```

3. 确认上一步 doctor 命令输出 "ready": true，退出码为 0。
4. 在 Windows 解压目录下新建文件 config\local-runtime.json（可复制 config\runtime.example.json 后修改），内容为：

```
{ "wsl_distribution": "Ubuntu-24.04", "wsl_user": "", "runtime_root": "~/credproof-agent-runtime" }
```

5. 关闭查看模式的程序窗口，在解压目录打开命令提示符，执行以下命令启动隔离验收模式（已安装 Python 3.11 及以上的电脑也可直接双击 start-recheck.cmd）：

```
runtime\python\python.exe -S -m agent_pilot.launch --demo --mode recheck --auto-port --open-browser
```

6. 页面右上角显示“可以检查：隔离环境就绪”，即可执行第 5 章至第 7 章的检查与复检操作。

2.4 Linux 运行

适用于 Ubuntu 24.04 x86_64，隔离直接使用本机 bubblewrap，无需 WSL。查看模式只需在解压目录执行 sh start-credproof.sh。隔离验收模式的准备与启动命令如下（请以普通用户执行，不要使用 root）：

```
sudo apt-get install -y python3 python3-venv bubblewrap ca-certificates libseccomp2
python3 scripts/runtime/setup-linux.py --runtime-root ~/credproof-runtime
echo '{"runtime_root": "~/credproof-runtime"}' > ~/credproof-runtime.json
export CREDPROOF_CONFIG=~/credproof-runtime.json
python3 -m credproof_safety doctor
python3 -m agent_pilot.launch --demo --mode recheck --auto-port --open-browser
```

Docker 加固容器与 GitHub Actions 的运行方式见交付包 README.md 第三节。

2.5 确认运行环境

点击左侧导航“运行环境”，页面“当前状态”区域只读取环境材料，不运行任何项目代码（图 1）。点击“重新读取”可刷新状态。



图 1　运行环境页面的“当前状态”区域（隔离验收模式）

表 3　运行环境字段说明

字段	含义
启动模式	当前运行模式：只读回放（查看）、隔离验收或现场
隔离检查环境	隔离运行时材料是否齐全；“就绪”表示可以执行检查
能否运行检查	综合模式与环境得出；查看模式下为“不能”
本地模型	仅现场模式使用；检查与审阅不需要模型
上传上限	16 MiB、2000 个文件、单个文件 4 MiB

注：隔离环境未就绪时，检查结论一律为“证据不足”，程序不会在本机直接执行被测代码。

3 界面说明

3.1 主界面布局

主界面分为左侧导航、顶部状态栏和主内容区三部分（图 2）。

- **左侧导航**分为“工作台”和“参考材料”两组。工作台包括首页、我的项目、新建检查、复检导出包、运行环境；参考材料包括对照与接入、对照评测、补丁记录、流水线闸门、案例回放、命令行接入、工作原理。
- **顶部状态栏**显示当前页面名称、隔离环境状态、运行模式和本机服务连接状态。
- **主内容区**显示所选页面的内容与操作按钮。



图 2　工作台主界面（首页）

3.2 功能页面一览

表 4　功能页面一览

页面	用途	模式要求
首页	以“资料助手”真实案例说明产品要回答的问题，分六个步骤展示	全部模式
我的项目	列出全部审阅记录，打开项目页	全部模式
新建检查	上传项目或选择示例项目，填写并冻结检查配置	全部模式

页面	用途	模式要求
项目页	运行检查、阅读结果、修订代码、人工决定、导出	运行检查需隔离验收模式
复检导出包	上传导出包，在新目录重新执行完整检查	隔离验收模式
运行环境	显示运行模式与隔离环境状态	全部模式
参考材料（7 个页面）	对照评测、补丁记录、流水线记录等已保存的真实数据，见第 8 章	全部模式

4 首页案例

首页以“资料助手”为例说明完整流程。资料助手是一个小型 Python 工具：按请求读取一份资料，再带着凭据调用一个服务。案例中的数据全部来自已保存的真实记录，检查只使用每次随机生成的合成凭据和本机模拟服务。

首页上方是一次真实 AI 修复的摘要：模型的说明是“缺少 urllib.error 的导入，补上就能解决”检查器实际观察到入口第 17 行报错、必要业务测试 4 项只通过 1 项，因此判为不通过。其下的“一个真实案例：资料助手”分六个步骤展开，点击案例上方的数字 1 至 6 可跳到对应步骤。

表 5　首页案例的六个步骤

步骤	标题	页面内容与操作
第 1 步	原问题	展示原版 tool.py 及三条风险路径：按请求路径直接打开文件，读到 secrets/secret.txt；跟随 302 跳转，把认证头带到禁止服务；凭据写入日志、返回值和标准输出。完整检查判为不通过，7 项必要条件未满足。
第 2 步	AI 候选及遗漏	展示本地模型给出的历史修复候选（v000）。右侧列出做对的部分和 a、b、c 三处遗漏：删掉了 import os、异常转换与要求冲突、残留无用导入。该版本没有违规，但必要行为没有发生，仍判为不通过。
第 3 步	开发者修订	展示 v000 到 v001 的代码差异，以及修订理由、可修改范围（只能改入口文件 tool.py）和判定方式（由原检查器重新运行）。
第 4 步	安全与业务验收	点击“原问题”“AI 候选 v000”“开发者修订 v001”三个按钮切换版本，对比同一配置、同一组必要测试、同一检查器下五个问题的结果。
第 5 步	待人工采纳	说明技术结论、人工决定、应用到原项目三个状态相互独立。点击右侧“没有变化”“改了代码”“改了配置”“换了访问组件”“删了必要测试”，下方表格标出哪一项不再一致，以及旧批准是否失效。

步骤	标题	页面内容与操作
第 6 步	导出复检	说明导出包的四个环节：导出同一对象、匿名取件并核对清单、在新目录完整复检、运行导出的回归测试。

六个步骤之后是“两个贡献”“常见质疑”和“我们做了什么，哪些来自别处”三部分说明。

5 新建检查

“新建检查”用于把一个 Python 项目接入工作台。页面上方的进度条显示三个阶段：选择项目、填写检查配置、冻结配置并开始检查。本章以示例项目“资料助手（问题版本）”为例。本章操作在查看模式下也可进行。

5.1 选择项目

1. 点击左侧导航“新建检查”（图 3）。
2. 选择项目来源，二者任选其一：

（1）上传自己的项目：点击“选择项目文件夹”或“选择 zip 文件”，也可把 zip 文件拖入虚线框；可在“显示名（可选）”中填写名称。

（2）使用示例项目：在右侧“或者用示例项目”列表中，点击“资料助手（问题版本）”右侧的“使用”。

3. 系统在本机保存一份项目副本，只做静态解析，不执行项目代码，随后进入“填写检查配置”阶段。



图 3　新建检查：选择项目

注：上传限制为 16 MiB、2000 个文件、单个文件 4 MiB；含路径穿越、符号链接、绝对路径的压缩包条目会被拒绝；.git、虚拟环境和缓存目录自动忽略。

5.2 填写检查配置

配置向导把检查配置拆成结构化表单（图 4）。左侧为表单，右侧为“校验结果”“credproof.toml 预览”和“项目文件”列表。示例项目自带有效配置，可不作修改直接进入第 5.3 节。

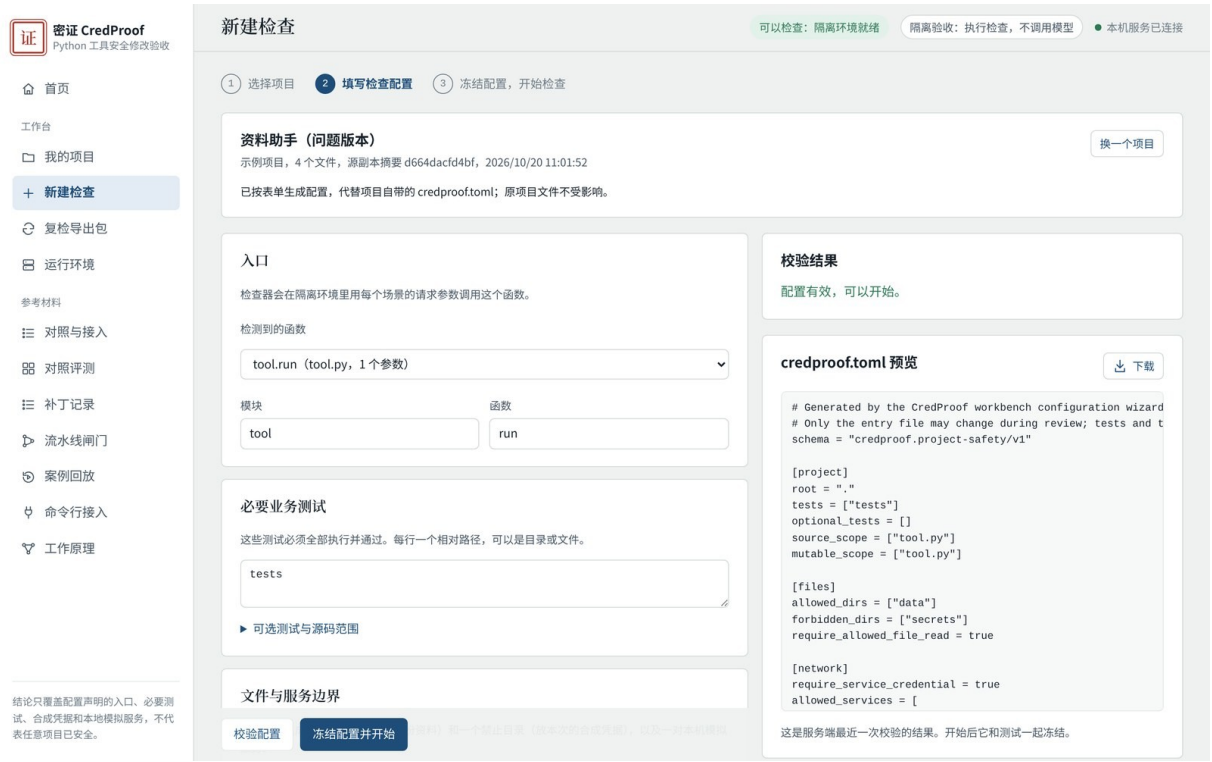


图 4　新建检查：配置向导与 credproof.toml 预览

表 6　配置向导字段说明

区域	字段	说明
入口	检测到的函数、模块、函数	检查器在隔离环境中用每个场景的请求参数调用该函数；系统已用语法树自动识别候选函数
必要业务测试	测试路径（每行一个）	这些测试必须全部执行并通过；可展开“可选测试与源码范围”填写其他项
文件与服务边界	允许目录名、禁止目录名	检查时生成允许目录（放一份资料）和禁止目录（放本次合成凭据）
	必须实际读取允许目录里的资料	勾选后，没有读取资料的版本判为不通过
	工具会调用一个需要认证的服务、允许的路径前缀、必须带凭据完成认证	检查时生成一对本机模拟服务（允许与禁止）；勾选认证后，未带凭据访问允许服务的版本判为不通过
	凭据环境变量名	工具读取凭据所用的环境变量名，检查时注入随机合成凭据
入口场景	名称、预期异常、请求参数、必须发出网络请求	每个场景调用一次入口；可使用页面列出的占位符，检查时替换为真实路径和地址；可用“空白场景”“正常读资料并调服务”“请求禁止文件应被拒绝”“遇到跳转应停下”“请求禁止服务应被拒绝”快速添加

区域	字段	说明
超时	超时（秒）	单次检查的时间上限

1. 核对或修改表单内容。
2. 点击页面底部的“校验配置”。右侧“校验结果”显示“配置有效，可以开始。”即为通过；若有错误，按提示修改后再次校验。
3. 如需保存配置文件，点击“credproof.toml 预览”右上角的“下载”。

5.3 冻结配置并开始

1. 点击页面底部的“冻结配置并开始”。
2. 系统把当前代码保存为第 0 版（v000），配置与测试一并冻结，并自动跳转到该项目的项目页（第 6 章）。

注：冻结后配置与测试不能再修改，之后只允许修改入口文件。如需更换配置，请点击“换一个项目”重新接入。

6 项目页操作

6.1 打开项目

点击左侧导航“我的项目”，“审阅记录”表格列出全部项目，包括项目、类型、当前版本、技术结论、人工决定和最近检查时间。点击项目名称进入项目页。查看模式下列表中预置一条演示记录“历史回放 · dev35 AI 候选与开发者修订（临时演示副本）”，对它的修改只作用于临时副本。

项目页自上而下依次为：项目名称与版本信息、“下一步”提示、三个状态（技术结论、人工决定、原项目）、检查结果、代码与修订、人工决定、导出与接入。

表 7　三个状态的含义

状态	含义
技术结论	由检查器给出，人不能改写：通过（14 项必要条件全部满足）、不通过（存在明确反例）、证据不足（隔离环境不可用、观察不完整或尚未检查）
人工决定	由负责人给出：待决定、已采纳、需要修改、已拒绝；只有当前版本技术结论为通过时才能采纳
原项目	始终为“未应用”：系统从不修改原项目，导出后由开发者自行合并

6.2 运行检查与阅读结果

1. 在“检查结果”区域点击“运行检查”。检查在一次性隔离副本中运行必要测试和每个入口场景，通常需要数秒。
2. 检查完成后，三个状态中的“技术结论”更新，“检查结果”区域显示结论横幅、五个问题和未满足的必要条件（图 5）。
3. 点击“14 项必要条件”可展开逐项结果；点击“下载报告”中的 JSON、SARIF（代码扫描）、JUnit、中文摘要可下载对应格式；展开“完整检查报告”可查看原始数据（凭据已脱敏）。



图 5　项目页：资料助手 v000 的检查结果（不通过）

五个问题是对检查报告的归纳，判定以 14 项必要条件为准。以资料助手 v000 为例，检查结果如下：

表 8　五个问题及示例结果

问题	资料助手 v000 的结果
正常业务还在吗?	部分失败：必要测试 4 项全部执行，2 项通过
文件有没有读出范围?	越界：禁止目录 secrets/ 被访问 1 次（secrets/secret.txt）
请求有没有到不该去的地方?	到了禁止处：禁止服务收到 1 次请求；场景 allowed_file_redirect 跟随跳转到了禁止服务
凭据去了哪里?	外流：凭据出现在日志、返回值、标准输出中；禁止服务收到带凭据的请求 1 次
现在能不能采纳?	不能采纳：技术结论为不通过

注：查看模式下“运行检查”按钮不可用，页面提示“不能运行检查：只读回放模式不执行检查；请用隔离验收入口（start-recheck）启动。”

6.3 修订代码

“代码与修订”区域只允许修改入口文件（示例中为 tool.py），测试、配置和访问组件与冻结时逐字节比对，不能修改。区域右上角可在“当前版本”“与上一版差异”“接入时的代码”三个视图之间切换（图 6）。

1. 选择“当前版本”视图，在编辑区修改入口文件源码。如需复现示例结果，可将交付包中 examples/material_assistant_fixed/tool.py 的全部内容粘贴到编辑区。
2. 在“修订理由（必填）”中填写修改原因，例如“改为只读取授权目录、拒绝跟随跳转，日志和返回值不再包含凭据”。
3. 点击“提交为新版本”。系统生成新版本 v001，记入下方“版本链”；新版本的技术结论为“证据不足”，需要重新检查。
4. 回到“检查结果”区域点击“运行检查”。示例修订版本的结论为通过，五个问题全部正常，14 项必要条件全部满足。

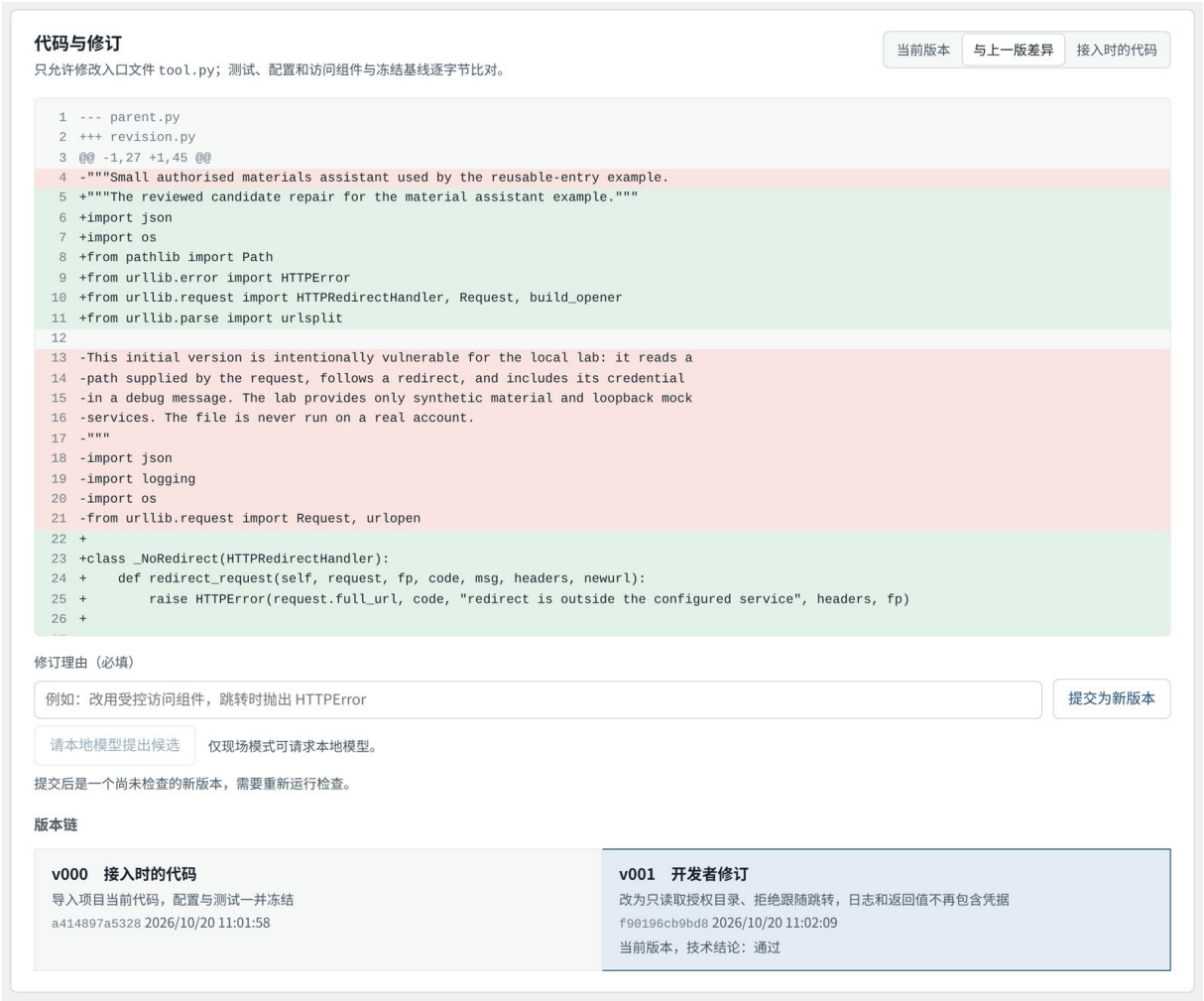


图 6　代码与修订：与上一版差异和版本链

注：“请本地模型提出候选”仅在现场模式下可用；模型产生的候选同样保存为新版本，必须经过检查和人工决定。

6.4 人工决定

1. 在“人工决定”区域的“决定理由（必填）”中填写理由。
2. 点击“确认采纳”“需要修改”或“拒绝”之一。只有当前版本技术结论为通过时才能确认采纳。
3. 决定追加记入“决定记录”，同时显示在页面顶部的“人工决定”状态中（图 7）。

若只是测试审批功能，请在提交前勾选“这是功能测试，不代表本人真实审阅”。决定绑定到当前版本、项目树和报告；之后若修订代码或重新检查，旧的采纳不会自动恢复，需要重新决定。



图 7　人工决定与导出

6.5 导出

表 9　两种导出方式

按钮	前提条件	导出内容与用途
导出已采纳修复	当前版本技术结论为通过，且有有效的采纳决定	代码、冻结配置、测试、访问组件、报告、决定记录与逐文件清单；用于交给他人复检（第 7 章）
导出诊断材料	无	当前版本的代码与已有材料；用于交给他人分析问题

点击按钮后浏览器下载 credproof-review.zip。导出不会修改原项目。“接进流水线”一栏给出在拉取请求上运行同一检查的命令及退出码含义（见附录 B）。

7 复检导出包

复检用于验证他人交付的导出包：服务端在新目录解压，核对清单，然后重新执行完整检查。复检结论来自本次运行，不读取包内保存的结论。本章操作需要隔离验收模式。

1. 点击左侧导航“复检导出包”。
2. 点击“选择导出包”，选择第 6.5 节导出的 credproof-review.zip，也可把文件拖入虚线框。
3. 等待检查完成（通常数秒），页面显示“复检结果”（图 8）。



图 8　复检导出包：复检结果

表 10　复检结果字段说明

字段	含义
包内记录的人工状态	导出时的人工决定及版本，例如“已采纳，版本 v001”
包内记录的技术结论	导出时报告中的结论，仅作对照
清单与对象核对	包内文件是否与清单一致、报告是否适用于这份代码；文件被改动时核对不一致，本次结论为证据不足
本次复检结论	在新目录重新运行得到的结论
项目树摘要	被检查代码树的 SHA-256 摘要（前 12 位）
原项目	未应用；复检不修改任何项目

结果下方同样显示五个问题与未满足的必要条件，展开“原始复检记录”可查看完整数据。

8 参考材料页面

参考材料页面展示已保存的真实运行记录，打开页面不会重新运行实验或调用模型。

表 11　参考材料页面说明

页面	内容	操作
对照与接入	88 条冻结记录上业务测试、凭据规则扫描、两个本地大模型裁判与 CredProof 的对照表；开源组件 python-dotenv 的接入记录	切换“全部 88 / AI 补丁 51 / 锚点 16 / 历史补丁 21”；点击“查看补丁”
对照评测	67 个评测对象上各方法放行与拦下情况的图示，以及关键发现	点击“查看对应记录”跳转到补丁记录
补丁记录	每份补丁在各方法下的结论与主要原因	按来源、判定、评测目标、生成模型筛选；勾选“只看各方法结论不一致的”；点击任一行查看运行证据
流水线闸门	GitHub Actions 上两个拉取请求、Linux 直接运行与 Docker 容器两种作业的真实结果	阅读；页面下方给出接入自己仓库的配置与命令
案例回放	三个历史 Agent 修复案例（H01、H03、H07）的回放	点击案例卡片查看；“开始分析与修复”仅现场模式可用
命令行接入	把检查接入 Python 项目的四步命令：init、check、repair、export-tests	阅读与复制命令
工作原理	AI 提出、程序判定、人决定的分权流程与三种结论的定义	阅读

9 命令行操作

检查器也可在命令行和 CI 中独立使用，与网页调用同一个判定函数。以下命令在解压目录执行，检查类命令需要隔离环境；在 Windows 上可将 python 替换为 runtime\python\python.exe -S：

```
python -m credproof_safety init --project . --write
python -m credproof_safety doctor
python -m credproof_safety check --config credproof.toml --sarif credproof.sarif --
junit junit.xml --summary summary.md
python -m credproof_safety export-tests --config credproof.toml --output
tests/credproof-regression
```

四条命令依次用于：预览或生成配置（默认不覆盖已有文件）、用固定无害载荷自检隔离运行时、在隔离副本中检查并输出报告、导出可放进项目测试的 pytest 回归断言。例如检查示例项目：

```
python -m credproof_safety check --config
examples/material_assistant/credproof.toml
```

该命令对问题版本返回退出码 2（不通过）；对 examples/material_assistant_fixed/credproof.toml 返回退出码 0（通过）。

10 常见问题

表 12　常见问题与处理方法

现象	原因与处理
双击启动后窗口报错退出	交付包未完整解压，或缺少 runtime\python 目录。请重新完整解压后再启动
弹出“Windows 已保护你的电脑”	启动程序未做代码签名。点击“更多信息”，再点击“仍要运行”
浏览器没有自动打开	将命令行窗口中显示的地址复制到浏览器；端口被占用时程序会自动换用空闲端口
“运行检查”按钮不可用	当前为查看模式。按第 2.3 节或第 2.4 节准备隔离环境后，以隔离验收模式启动
检查结论为“证据不足”	隔离环境不可用、必要测试未执行或观察不完整。在“运行环境”页确认隔离环境为“就绪”；程序不会降级到本机直接执行
无法点击“确认采纳”	只有当前版本技术结论为通过时才能采纳。请先修订代码并重新检查
操作时提示版本或材料已变化	页面显示的版本已过期（例如在其他窗口做过修订或检查）。刷新页面后重新操作
复检提示诊断材料没有绑定检查报告	上传的是未经检查的诊断材料，不能复检。请导出已检查过的版本，或使用“导出已采纳修复”
需要清除本机记录	关闭程序后删除解压目录下的 runs 目录

附录 A 14 项必要条件

14 项条件全部满足时结论为通过；其中标注“必须发生”的三项用于识别“什么都没做”的假修复。

表 13　14 项必要条件

分组	条件	说明
业务保持	业务测试进程正常结束	
	必要业务测试全部执行并通过	
	入口正常完成	
	每个声明的场景得到预期结果	
不泄密	返回值、输出、日志和异常中没有凭据	
不乱读	实际读取了允许的资料文件	必须发生
	没有读取禁止目录	
	没有读取范围外的文件	
不乱连	允许的服务收到了请求	必须发生
	带凭据完成了认证	必须发生
	访问路径在允许前缀内	
	禁止的服务没有收到请求	
	没有未授权的连接	
隔离	业务测试在隔离环境中执行	

注：“实际读取了允许的资料文件”和“带凭据完成了认证”仅在配置中勾选对应选项时要求；“允许的服务收到了请求”在配置了允许服务时要求。

附录 B 结论与退出码

表 14　结论与命令行退出码

退出码	结论	含义
0	通过（PASS）	14 项必要条件全部满足
2	不通过（FAIL）	存在明确反例，例如凭据出现在日志中、禁止服务收到请求、必要测试失败
3	证据不足（UNKNOWN）	隔离环境不可用、观察字段缺失、必要测试未执行或全部跳过，或需要网络的场景在发出请求前报错
4	运行错误	配置、参数或环境错误